```
import os, zipfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report,
ConfusionMatrixDisplay, roc_auc_score

tf.random.set_seed(42)

# COLAB
FILE_ID = "1inGuiluh0PDy0O7j19NAJLQNdTNrzo1T"
!gdown {FILE_ID} -O /content/creditcard.zip
with zipfile.ZipFile("/content/creditcard.zip") as z:
    z.extractall("/content/data")
print(os.listdir("/content/data"))
CSV = "/content/data/Credit Card Dataset/creditcard.csv"        #
adjust if the printed name or folder differs

# JUPYTER (exam day): data already extracted, so skip the lines above
and use the full path of the csv
# CSV = "/home/student/Desktop/creditcard.csv"
#https://drive.google.com/file/d/1inGuiluh0PDy0O7j19NAJLQNdTNrzo1T/
view?usp=drive_link

Downloading...
From (original): https://drive.google.com/uc?
id=1inGuiluh0PDy0O7j19NAJLQNdTNrzo1T
From (redirected): https://drive.google.com/uc?
id=1inGuiluh0PDy0O7j19NAJLQNdTNrzo1T&confirm=t&uuid=ed6ae4cc-985c-
4f6e-861c-f0240b10857a
To: /content/creditcard.zip
100% 69.0M/69.0M [00:01<00:00, 39.9MB/s]
['Credit Card Dataset']

df = pd.read_csv("/content/data/Credit Card Dataset/creditcard.csv")
print(df.shape)
print(df["Class"].value_counts())

(284807, 31)
Class
0    284315
1       492
Name: count, dtype: int64

X = df.drop(columns=["Time", "Class"]).values
y = df["Class"].values
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=42)

scaler = StandardScaler().fit(X_train[y_train == 0])
X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)

X_train_normal = X_train[y_train == 0]
print(X_train_normal.shape, X_test.shape, "| frauds in test:",
y_test.sum())
```

```
(227451, 29) (56962, 29) | frauds in test: 98
```

```
model = keras.Sequential([
    layers.Input(shape=(29,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu"),        # bottleneck
    layers.Dense(16, activation="relu"),
    layers.Dense(29),                          # reconstruct the 29
features
])
model.summary()
model.compile(optimizer="adam", loss="mse")

history = model.fit(X_train_normal, X_train_normal,
validation_split=0.1, epochs=20, batch_size=256, verbose=2)

plt.plot(history.history["loss"], label="train loss")
plt.plot(history.history["val_loss"], label="val loss")
plt.legend()
plt.show()
```

```
Model: "sequential_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ dense_1 (Dense)                      │ (None, 16)                  │
480 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense_2 (Dense)                      │ (None, 8)                   │
136 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense_3 (Dense)                      │ (None, 16)                  │
144 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
```

```
──────┤
│ dense_4 (Dense)                 │ (None, 29)             │
493 │
└─────────────────────────────────┴────────────────────────┴──────────
──────┘

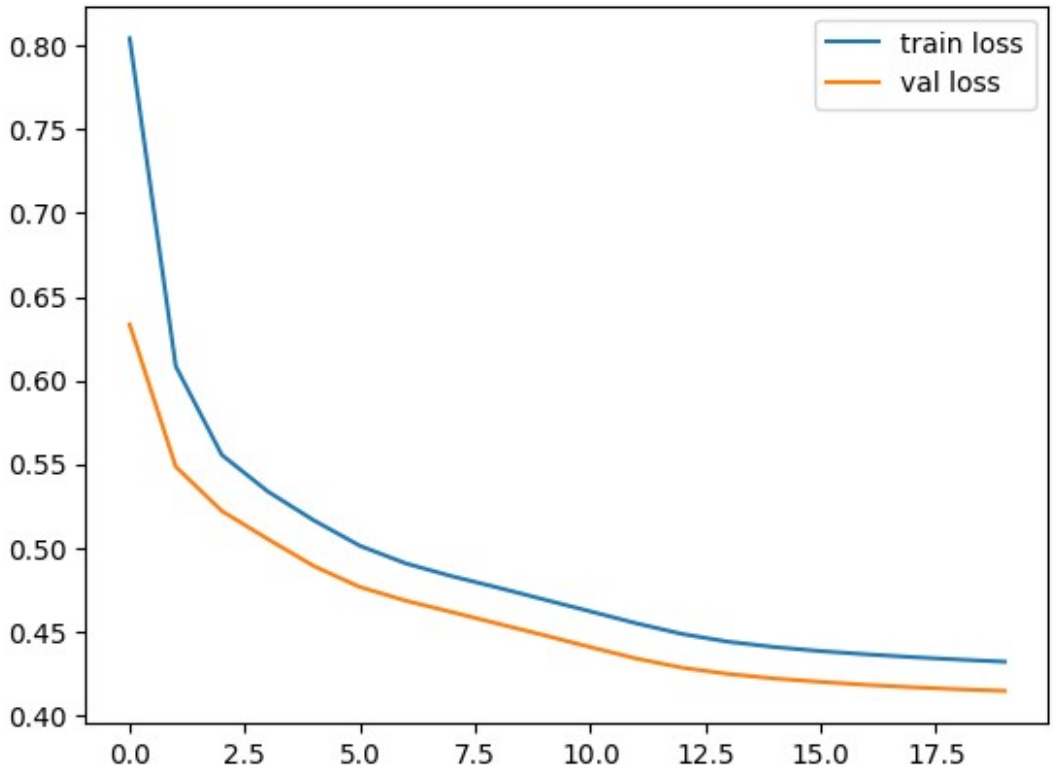
 Total params: 1,253 (4.89 KB)

 Trainable params: 1,253 (4.89 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/20
800/800 - 6s - 7ms/step - loss: 0.8044 - val_loss: 0.6333
Epoch 2/20
800/800 - 3s - 4ms/step - loss: 0.6085 - val_loss: 0.5485
Epoch 3/20
800/800 - 2s - 2ms/step - loss: 0.5555 - val_loss: 0.5219
Epoch 4/20
800/800 - 3s - 3ms/step - loss: 0.5336 - val_loss: 0.5051
Epoch 5/20
800/800 - 2s - 2ms/step - loss: 0.5163 - val_loss: 0.4890
Epoch 6/20
800/800 - 2s - 2ms/step - loss: 0.5011 - val_loss: 0.4765
Epoch 7/20
800/800 - 6s - 7ms/step - loss: 0.4905 - val_loss: 0.4683
Epoch 8/20
800/800 - 5s - 6ms/step - loss: 0.4829 - val_loss: 0.4615
Epoch 9/20
800/800 - 3s - 3ms/step - loss: 0.4761 - val_loss: 0.4545
Epoch 10/20
800/800 - 2s - 2ms/step - loss: 0.4690 - val_loss: 0.4476
Epoch 11/20
800/800 - 3s - 3ms/step - loss: 0.4619 - val_loss: 0.4406
Epoch 12/20
800/800 - 2s - 2ms/step - loss: 0.4548 - val_loss: 0.4337
Epoch 13/20
800/800 - 2s - 3ms/step - loss: 0.4484 - val_loss: 0.4282
Epoch 14/20
800/800 - 2s - 2ms/step - loss: 0.4438 - val_loss: 0.4245
Epoch 15/20
800/800 - 2s - 2ms/step - loss: 0.4406 - val_loss: 0.4218
Epoch 16/20
800/800 - 2s - 2ms/step - loss: 0.4382 - val_loss: 0.4198
Epoch 17/20
800/800 - 2s - 3ms/step - loss: 0.4363 - val_loss: 0.4180
Epoch 18/20
800/800 - 2s - 3ms/step - loss: 0.4346 - val_loss: 0.4166
Epoch 19/20
800/800 - 2s - 2ms/step - loss: 0.4331 - val_loss: 0.4154
```

```
Epoch 20/20
800/800 - 2s - 2ms/step - loss: 0.4318 - val_loss: 0.4144
```




```
def recon_error(data):
    return np.mean((data - model.predict(data, verbose=0)) ** 2,
axis=1)      # per-transaction error

err_train = recon_error(X_train_normal)
err_test = recon_error(X_test)

threshold = np.percentile(err_train, 99)          # flag anything
worse than 99% of normal transactions
y_pred = (err_test > threshold).astype(int)

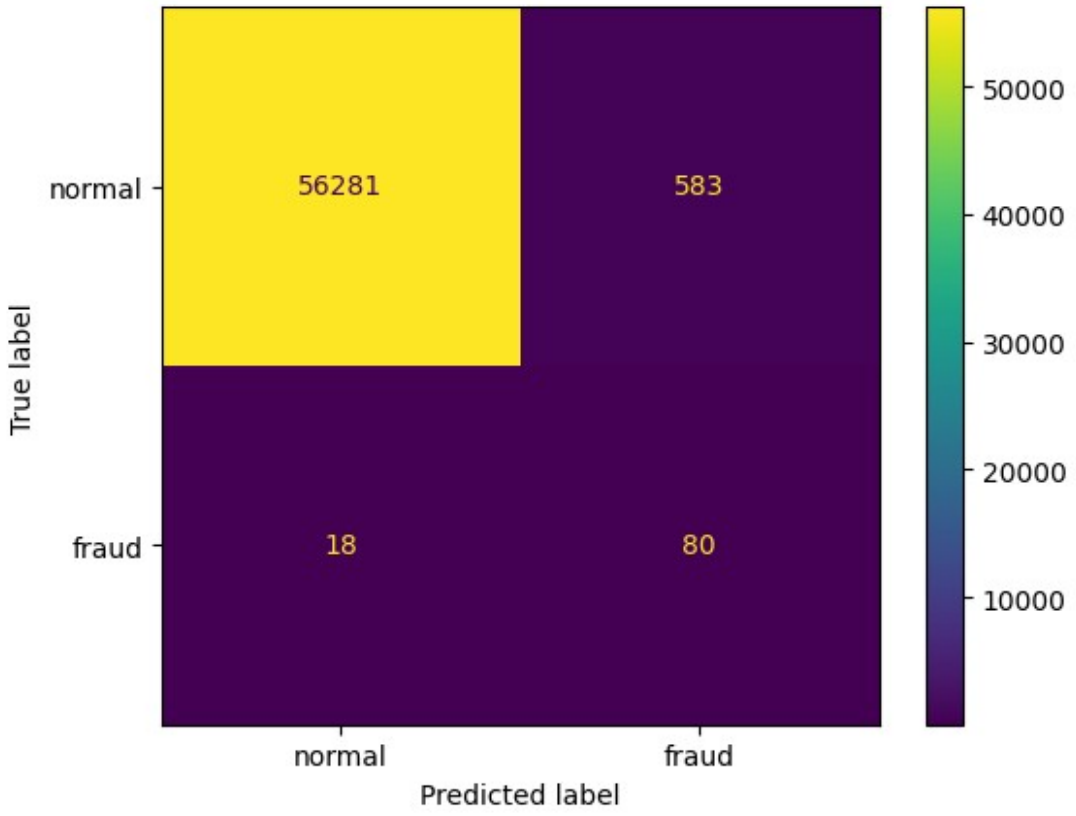
print("Threshold:", threshold)
print("ROC AUC:", roc_auc_score(y_test, err_test))
print(classification_report(y_test, y_pred, target_names=["normal",
"fraud"]))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
display_labels=["normal", "fraud"])
plt.show()

Threshold: 1.9224333128164557
ROC AUC: 0.9485471242520643
              precision    recall  f1-score   support
```

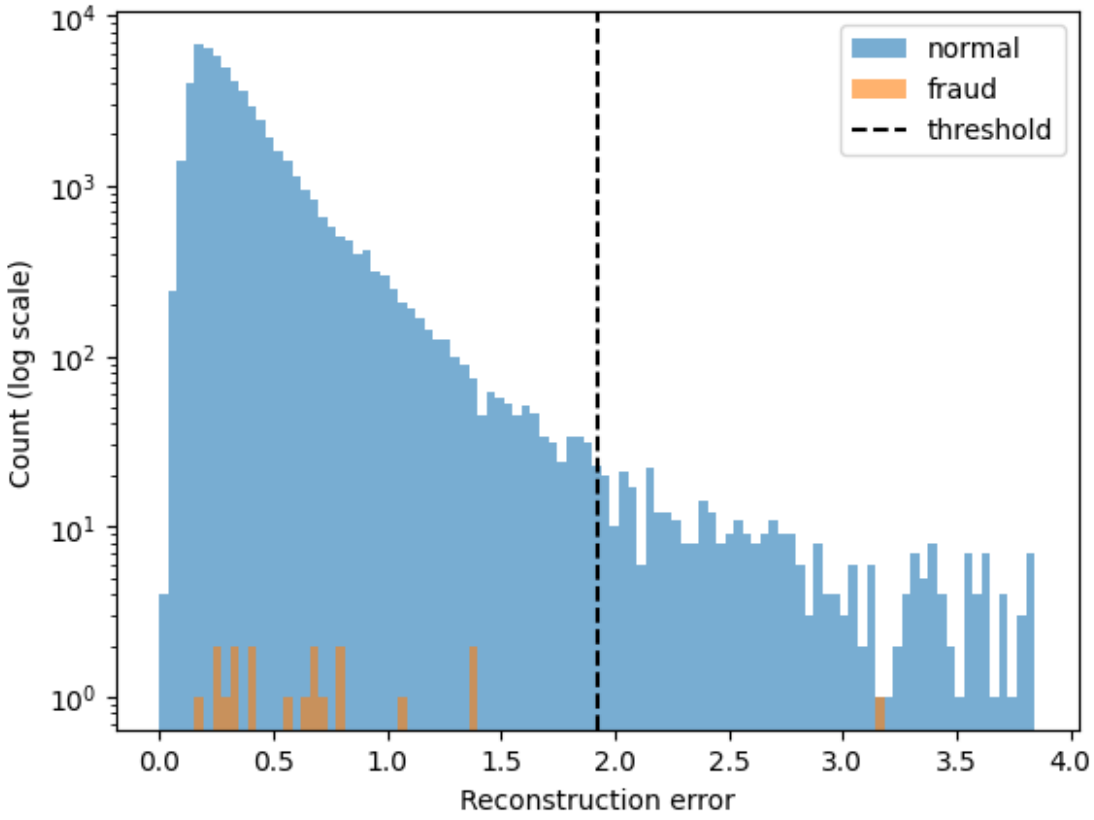
```
              precision    recall  f1-score   support

      normal       1.00      0.99      0.99     56864
       fraud       0.12      0.82      0.21        98

    accuracy                           0.99     56962
   macro avg       0.56      0.90      0.60     56962
weighted avg       1.00      0.99      0.99     56962
```




```
bins = np.linspace(0, np.percentile(err_test, 99.5), 100)
plt.hist(err_test[y_test == 0], bins=bins, alpha=0.6, label="normal",
log=True)
plt.hist(err_test[y_test == 1], bins=bins, alpha=0.6, label="fraud",
log=True)
plt.axvline(threshold, color="k", linestyle="--", label="threshold")
plt.xlabel("Reconstruction error")
plt.ylabel("Count (log scale)")
plt.legend()
plt.show()
```


normal
fraud
threshold
Count (log scale)
10^4
10^3
10^2
10^1
10^0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Reconstruction error